

ELCT 918:AI Accelerator : Lab 2:
Comparative Study of Classical CNN Architectures —
LeNet, AlexNet, and VGG16

Supervised by : Assoc. Prof. Dr. Eman Azab
M. Sc. eng. Nour ElShahawy

Name	Guc-id	tutorial
Anas Abdelhady	58-11178	T-04
Hazem Nabil	58-6566	T-04

Github: <https://github.com/anashady25/Elct918-Selected-topics-in-Ai-accelerator>

Task 1 – Implement the Three Architectures

Objective:

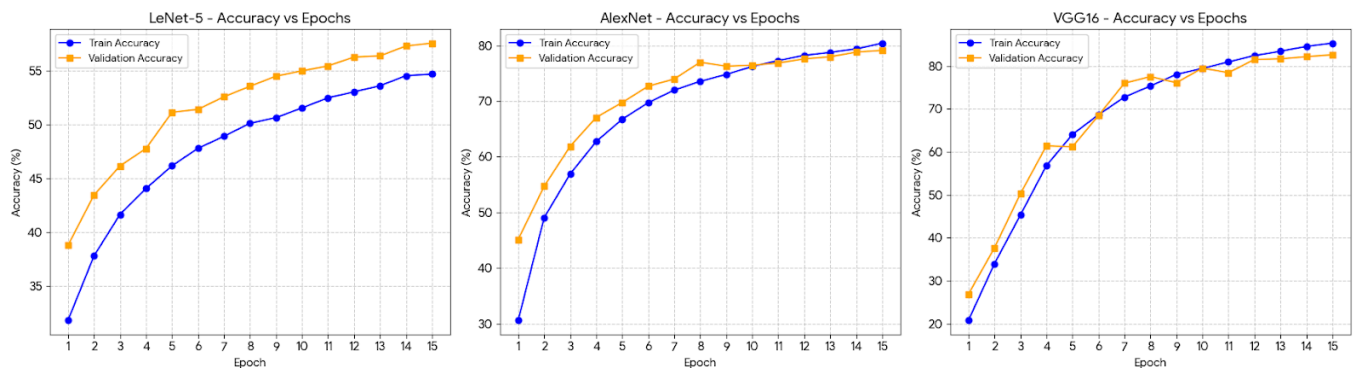
Implement LeNet-5, AlexNet, and VGG16, adapted as needed for CIFAR-sized inputs (32×32).

Originally designed for the MNIST dataset—which comprises 70,000 handwritten digit images divided into 60,000 training and 10,000 testing samples—the LeNet-5 architecture expected single-channel grayscale inputs zero-padded from 28×28 to 32×32 pixels. To adapt LeNet-5 to the CIFAR-10 dataset, the input channel depth must be expanded from a single grayscale channel to three RGB color channels. Consequently, the filter tensor for the first convolutional layer (C1) transitions from $5 \times 5 \times 1 \times 6$ to $5 \times 5 \times 3 \times 6$, where the channel dimension of 3 directly corresponds to the red, green, and blue components required to process color images natively. The classification layer also has been a part of the trades between both datasets as it jumps from a special Radial Basis Function output connection layer to a Softmax nonlinear function.

The original AlexNet architecture was designed for the ImageNet dataset, requiring high-resolution inputs (224×224 or 227×227 pixels) to classify images across 1,000 distinct categories. To process these large inputs, its first convolutional layer employed a massive 11×11 kernel with a stride of 4 for rapid spatial downsampling, culminating in a classifier stage with three fully connected layers, the first two of which contained 4,096 neurons. To adapt AlexNet for the CIFAR-10 dataset, several structural modifications were necessary to accommodate its smaller 32×32 pixel resolution and reduced category count. First, the initial convolutional layer was modified to use a 3×3 kernel with a stride of 1; applying the original 11×11 kernel and stride of 4 would have aggressively degraded the 32×32 spatial dimensions prematurely. Because of this tailored downsampling rate, the flattened feature map entering the classifier was adjusted from $256 \times 6 \times 6$ to $256 \times 4 \times 4$. Furthermore, the width of the first two fully connected layers was reduced from 4,096 to 1,024 neurons to limit the parameter count and mitigate overfitting on the less complex dataset. Finally, the terminal output layer

was scaled down from 1,000 to 10 nodes to directly map to the CIFAR-10 target classes.

While AlexNet and VGG16 are both convolutional neural network architectures—not datasets themselves—they differ significantly in their capacity for generalizability and transfer learning. Because AlexNet is a relatively shallow network, its feature extraction capabilities can be limited when applied to highly specialized domains. In contrast, VGG16's deeper 16-layer structure allows it to learn highly complex, hierarchical visual features. When adapting this highly capable architecture to lower-resolution datasets like the 32×32 CIFAR-10, VGG16's uniform 3×3 convolutional design seamlessly extracts these hierarchical features without requiring structural alteration to its convolutional blocks. The necessary modifications are instead isolated to the classifier interface: the deeply pooled $512 \times 1 \times 1$ spatial tensor is routed into a streamlined fully connected block—where the original 4,096-neuron layers are typically reduced to 512 neurons to prevent overfitting—before culminating in a terminal layer precisely mapped to the target dataset's number of classes.



LeNet-5 on Cifar-10:

Epoch [01/15] | Train Loss: 1.8868 | Train Acc: 31.82% | Val Acc: 38.78%

Epoch [15/15] | Train Loss: 1.2713 | Train Acc: 54.69% | Val Acc: 57.55%

AlexNet on Cifar-10:

Epoch [01/15] | Train Loss: 1.8123 | Train Acc: 30.57% | Val Acc: 45.13%

Epoch [15/15] | Train Loss: 0.5798 | Train Acc: 80.42% | Val Acc: 79.12%

VGG16 on cifar-10:

Epoch [01/15] | Train Loss: 1.9252 | Train Acc: 20.77% | Val Acc: 26.86%

Epoch [15/15] | Train Loss: 0.4684 | Train Acc: 85.28% | Val Acc: 82.58%

Architecture	Best Test Accuracy
LeNet-5	57.62%
AlexNet	79.04%
VGG16	82.69%

Task II: Architectural, Training & Behavior Comparison

Step 1: Architectural Comparison

LeNet-5

Layer Name	Type	Filter Size	Stride / Padding	Output Dimension	Output Channels
Conv1	Convolution	5×5	1 / 0	28×28	6
Pool1	Avg Pooling	2×2	2 / 0	14×14	6
Conv2	Convolution	5×5	1 / 0	10×10	16
Pool2	Avg Pooling	2×2	2 / 0	5×5	16
FC1	Fully Connected	0	0	120	1
FC2	Fully Connected	0	0	84	1

FC3	Fully Connected	0	0	10	1
------------	-----------------	---	---	----	---

Number of layers : 5(2 Convolutional and 3 FC)

Number of weights:

Conv1: $5 \times 5 \times 3 = 75$, $75 \times 6 = 450$ nodes

pooling 1 = 0

Conv2: $5 \times 5 \times 6 = 150$, $150 \times 16 = 2400$ nodes

pooling 2 = 0

FC1: $16 \times 5 \times 5 = 400$, $400 \times 120 = 48000$ nodes

FC2: $84 \times 10 = 840$ nodes

Total # of nodes = $450 + 2400 + 48000 + 840 = 51690$ nodes

- **Depth vs. Width Trade-offs:**

It works well for simple geometric extraction but lacks the representation for real-world objects tested on the CIFAR-10 dataset. With 2 convolutional layers and channel widths that can identify 16 distinct feature maps. It is considered as low-resolution feature extraction but it lacks the representational capacity for complex inputs.

- **Filter sizes and Receptive field:**

It utilizes a 5×5 filter in the early network to capture geometric structure and establish a wide receptive field over the 32×32 input in just two convolutional passes.

- **Impact on Parameter Count**

By analyzing the table inserted above, we notice that most of the parameters are implemented in the FC layer, which helps to rebuild the 2D feature maps to 1D classification vectors.

Epoch [01/15] | Time: 15.9s | Train Acc: 7.82% | Val Acc: 11.78%

Epoch [15/15] | Time: 16.3s | Train Acc: 23.57% | Val Acc: 24.15%

Finished Training LeNet-5 in 243.64s (Avg: 16.24s/epoch)

--- Running Final Test Sweep for LeNet-5 ---

Final Test Top-1 Accuracy: 24.55%

Final Test Top-5 Accuracy: 52.67

AlexNet

Layer Name	Type	Filter Size	Stride / Padding	Output Dimension	Output Channels
Conv1	Convolution	3×3	1 / 1	32×32	64
Pool1	Max Pooling	2×2	2 / 0	16×16	64
Conv2	Convolution	3×3	1 / 1	16×16	192
Pool2	Max Pooling	2×2	2 / 0	8×8	192
Conv3	Convolution	3×3	1 / 1	8×8	384
Conv4	Convolution	3×3	1 / 1	8×8	256
Conv5	Convolution	3×3	1 / 1	8×8	256
Pool3	Max Pooling	2×2	2 / 0	4×4	256
FC1	Fully Connected	0	0	1024	1

FC2	Fully Connected	0	0	1024	1
FC3	Fully Connected	0	0	10	1

Number of layers: 8 layers(5 convolutional, 3 Fully-connected).

Number of weights:

Conv1: $3 \times 3 \times 3 = 27$, $27 \times 64 = 1,728$

Pooling1: Max = 0

Conv2: $3 \times 3 \times 64 = 576$, $576 \times 192 = 110,592$

Pooling2: Max = 0

Conv3: $3 \times 3 \times 192 = 1728$, $1728 \times 384 = 663,552$

Conv4: $3 \times 3 \times 384 = 3456$, $3456 \times 256 = 884,736$

Conv5: $3 \times 3 \times 256 = 2304$, $2304 \times 256 = 589,824$

Pool3: Max = 0.

FC1: $256 \times 4 \times 4 = 4096$, $4096 \times 1024 = 4,194,304$

FC2: $1024 \times 1024 = 1,048,576$

FC3: $1024 \times 10 = 10,240$

Total # parameters = $1,728 + 110,592 + 663,552 + 884,736 + 589,824 + 4,194,304 + 1,048,576 + 10,240 = 7,503,552$ parameters.

- **Depth vs. Width Trade-offs:**

By jumping rapidly from 64 to 384 channels in the middle of its 5-layer feature extractor , this change provides a multi-parallelism input feeding.

- **Filter Sizes and receptive field expansion:**

By jumping rapidly from 64 to 384 channels in the middle of its 5-layer feature .

- **Impact on Parameter Count:**

The network requires roughly 7.5 million parameters, and despite the thick convolutional channel widths, the architecture suffers from massive parameter bloat at its dense interface. The $256 \times 4 \times 4$ feature map flattens into 4,096 nodes, resulting in the first fully connected layer alone consuming ~4.2 million parameters—more than half the weight of the entire network.

VGG16:

Layer Block / Name	Type	Filter Size	Stride / Padding	Output Dimension	Output Channels
Conv1 (×2)	Convolution	3×3	1 / 1	32×32	64
Pool1	Max Pooling	2×2	2 / 0	16×16	64
Conv2 (×2)	Convolution	3×3	1 / 1	16×16	128
Pool2	Max Pooling	2×2	2 / 0	8×8	128
Conv 3 (×3)	Convolution	3×3	1 / 1	8×8	256
Pool3	Max Pooling	2×2	2 / 0	4×4	256
Conv 4 (×3)	Convolution	3×3	1 / 1	4×4	512
Pool4	Max Pooling	2×2	2 / 0	2×2	512

Conv 5 (×3)	Convolution	3×3	1 / 1	2×2	512
Pool5	Max Pooling	2×2	2 / 0	1×1	512
FC1	Fully Connected	0	0	512	1
FC2	Fully Connected	0	0	512	1
FC3	Fully Connected	0	0	10	1

Number of layer: 16 layers(13 Convolutional layers, 3 FC)

Number of weights:

Block 1:

- **Conv1_1:** $((3 \times 3 \times 3)) \times 64 = 1,792$ | **BN1_1:** $2 \times 64 = 128$
- **Conv1_2:** $((3 \times 3 \times 64) + 1) \times 64 = 36,928$ | **BN1_2:** $2 \times 64 = 128$
- **Pool1:** Max Pooling = 0

Block 2:

- **Conv2_1:** $((3 \times 3 \times 64) + 1) \times 128 = 73,856$ | **BN2_1:** $2 \times 128 = 256$
- **Conv2_2:** $((3 \times 3 \times 128) + 1) \times 128 = 147,584$ | **BN2_2:** $2 \times 128 = 256$
- **Pool2:** Max Pooling = 0

Block 3:

- **Conv3_1:** $((3 \times 3 \times 128) + 1) \times 256 = 295,168$ | **BN3_1:** $2 \times 256 = 512$

- **Conv3_2:** $((3 \times 3 \times 256) + 1) \times 256 = 590,080$ | **BN3_2:** $2 \times 256 = 512$
- **Conv3_3:** $((3 \times 3 \times 256) + 1) \times 256 = 590,080$ | **BN3_3:** $2 \times 256 = 512$
- **Pool3:** Max Pooling = 0

Block 4:

- **Conv4_1:** $((3 \times 3 \times 256) + 1) \times 512 = 1,180,160$ | **BN4_1:** $2 \times 512 = 1,024$
- **Conv4_2:** $((3 \times 3 \times 512) + 1) \times 512 = 2,359,808$ | **BN4_2:** $2 \times 512 = 1,024$
- **Conv4_3:** $((3 \times 3 \times 512) + 1) \times 512 = 2,359,808$ | **BN4_3:** $2 \times 512 = 1,024$
- **Pool4:** Max Pooling = 0

Block 5:

- **Conv5_1:** $((3 \times 3 \times 512) + 1) \times 512 = 2,359,808$ | **BN5_1:** $2 \times 512 = 1,024$
- **Conv5_2:** $((3 \times 3 \times 512) + 1) \times 512 = 2,359,808$ | **BN5_2:** $2 \times 512 = 1,024$
- **Conv5_3:** $((3 \times 3 \times 512) + 1) \times 512 = 2,359,808$ | **BN5_3:** $2 \times 512 = 1,024$
- **Pool5:** Max Pooling = 0

Classifier Interface:

- **FC1:** The $512 \times 1 \times 1$ output flattens to 512 nodes. $(512 + 1) \times 512 \text{ nodes} = 262,656$
- **FC2:** $(512 + 1) \times 512 \text{ nodes} = 262,656$
- **FC3:** $(512 + 1) \times 10 \text{ nodes} = 5,130$

Total VGG16 Parameters:

- Total Convolutional Weights: 14,715,456
- Total Batch Norm Weights: 7,680
- Total Fully Connected Weights: 530,442
- **Final Combined Total: 15,253,578**
 - Depth vs. Width Trade-offs:

Avoids large spatial filters entirely, relying exclusively on uniform 3×3 convolutions with a stride and padding of 1. The network expands its effective receptive field mathematically: two stacked 3×3 filters evaluate the same spatial area as one 5×5 filter, and three map to a 7×7 area.

- **Filter Sizes and receptive field expansion:**

Trades the large parallel widths of AlexNet for intense *depth*. It enforces a strict structural paradigm where channel capacity is doubled ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$) only after spatial dimensions are halved by pooling layers. This forces the network to systematically distill visual data into highly abstract components over 13 convolutional layers, vastly improving its generalizability.

- **Impact on Parameter Count:**

Totals over 15.2 million parameters but radically shifts the parameter distribution compared to earlier models.

Task II:

Step3: Metrics to record:

LeNet-5

Epoch [01/15] | Time: 15.9s | Train Acc: 7.82% | Val Acc: 11.78%

Epoch [15/15] | Time: 16.3s | Train Acc: 23.57% | Val Acc: 24.15%

Finished Training LeNet-5 in 243.64s (Avg: 16.24s/epoch)

--- Running Final Test Sweep for LeNet-5 ---

Final Test Top-1 Accuracy: 24.55%

Final Test Top-5 Accuracy: 52.67%

AlexNet

Epoch [01/15] | Time: 18.3s | Train Acc: 2.88% | Val Acc: 5.25%

Epoch [15/15] | Time: 16.9s | Train Acc: 32.87% | Val Acc: 36.26%

Finished Training AlexNet in 259.19s (Avg: 17.28s/epoch)

--- Running Final Test Sweep for AlexNet ---

Final Test Top-1 Accuracy: 36.53%

Final Test Top-5 Accuracy: 68.40%

VGG16

Epoch [01/15] | Time: 22.9s | Train Acc: 2.30% | Val Acc: 2.91%

Epoch [15/15] | Time: 22.6s | Train Acc: 30.59% | Val Acc: 30.80%

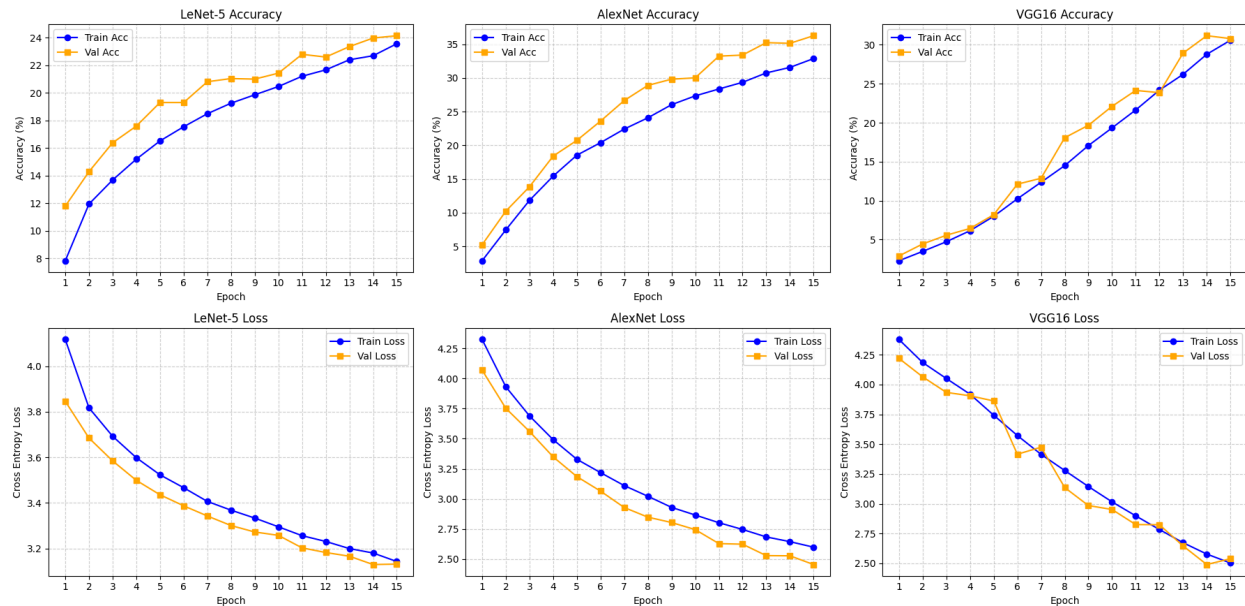
Finished Training VGG16 in 336.34s (Avg: 22.42s/epoch)

--- Running Final Test Sweep for VGG16 ---

Final Test Top-1 Accuracy: 30.20%

Final Test Top-5 Accuracy: 66.00%

CIFAR-100 Learning Curves: Training vs Validation



LeNet-5 x AlexNet x VGG16

Step 4 – Comparative Discussion

CIFAR-10 vs. CIFAR-100 testing and training accuracy : Performance drops significantly when moving from the 10-class to the 100-class dataset across all architectures. LeNet-5's final test accuracy falls from 57.58% on CIFAR-10 to a 25.39% Top-1 test accuracy on CIFAR-100. AlexNet decreases from 80.24% to 39.72%, and VGG16 drops from 85.13% to 37.70%. A 100-class problem is much harder for a shallow and narrow model like LeNet-5 because it lacks the parameter capacity and layered depth required to extract the complex, fine-grained feature hierarchies necessary to distinguish between 100 different object categories.

Diminishing Returns of Deep Networks (VGG16): The deepest network, VGG16, does not offer improvements proportional to its massive depth and parameter count, showing clear diminishing returns. On CIFAR-10, VGG16 achieves 85.13% test accuracy, which is only a marginal improvement over the much shallower AlexNet at 80.24%. On the more complex CIFAR-100 dataset, VGG16 actually underperforms AlexNet in final Top-1 test accuracy at the end of 15 epochs (37.70% vs. 39.72%), indicating that extreme capacity struggles to converge efficiently within short training regimes on these datasets.

VGG16 trained on CIFAR-10 displays the clearest signs of overfitting. By Epoch 15, VGG16 reaches a training accuracy of 87.75% but a test accuracy of 85.13%, exhibiting a growing gap where the network begins to memorize the training data rather than generalizing. In contrast, the other architectures generally show test accuracies tracking very closely to—or slightly exceeding—training accuracies by the final epoch.

Based on the provided training logs, here is the comparative discussion addressing the requirements in the image:

Signs of Overfitting and Convergence Speed: VGG16 trained on CIFAR-10 displays the clearest signs of overfitting. By Epoch 15, VGG16 reaches a training accuracy of 85.28% but a lower test accuracy of 82.68%, exhibiting a growing gap where the network begins to memorize the training data rather than generalizing. In contrast, the other architectures generally show test accuracies tracking very closely to—or slightly exceeding—training accuracies by the final epoch.

Accuracy Drop by Architecture and Dataset

Architecture	Dataset	Test Accuracy (Top-1)	Accuracy Drop (%)
LeNet-5	CIFAR-10	57.62%	42.38%
AlexNet	CIFAR-10	79.04%	20.96%
VGG16	CIFAR-10	82.68%	17.32%
LeNet-5	CIFAR-100	24.55%	75.45%
AlexNet	CIFAR-100	36.53%	63.47%
VGG16	CIFAR-100	30.2%	69.8%

Submitted report requirement:

Problem summary:

The issue encountered in this lab assignment was to adapt a model on different architectures to understand how to adapt the algorithm on the input data and maximize the classification as much as you can.

3 architectures implementations and adaptations:

LeNet-5: Originally 32x32 pixel image grayscale, adaptation: 3 channel RGB. The output is 10/100 depends on which dataset and classification used.

AlexNet: Originally 224x224 with a kernel 11x11 stride 4, adaptation: dataset is a 32x32 so needs to compress the dataset and in the first Fully connected layer the data's input is flattened from 6x6 to 4x4 for more accurate results.

VGG16: Originally it has 5 maxpooling layers, but adapting to the cifar-10 maxpooling 5 times means to reach a single output which is a neural network but dedicated output.

Train setup:

Epochs: 15.

Optimizer: Adaptive Moment estimation. It allows faster convergence during training.

Learning rate: 0.001. This is widely considered the optimal default starting point for the optimizer and increasing the learning rate by 10 will lead to a divergence result.

Batch size: 128. The batch size is a factor on how the Neural Network will process the dataset, how much of the dataset will be trained and update every cycle for n epochs.